ПРАКТИЧЕСКОЕ ЗАНЯТИЕ 01

Два приложения: GitHub Pages + Hugging Face Spaces

Создаём сайт, запускаем Python-сервер на собственном компьютере, публикуем браузерную версию приложения и сравниваем архитектуры.

РЕЗУЛЬТАТ ЗАНЯТИЯ

1. Публичная визитка **USERNAME.github.io** с двумя отдельными коммитами: HTML/CSS, затем JavaScript.
2. **Локальный Gradio**: Python-приложение открывается на localhost и перестаёт работать после остановки сервера.
3. **Gradio Lite**: аналогичная функция опубликована на бесплатном Hugging Face Static Space.
4. Студент понимает разницу между исполнением Python на сервере и в браузере.

Что подготовить

- Учётные записи GitHub и Hugging Face. Регистрацию на занятии не разбираем; доступ лучше проверить заранее.
- Windows 10/11, браузер, Git, редактор файлов, Python с пакетом Gradio. Способы подготовки — в отдельной технической памятке (включая вариант без прав администратора).
- Доступ к ИИ для генерации кода. Разрешена любая модель; работоспособность результата проверяется студентом.

ПРАВИЛО КУРСА

Исходники создаются и изменяются **локально**. Передача кода на GitHub и Hugging Face — только через **git add → git commit → git push**. Веб-интерфейсы разрешены для регистрации, создания пустого репозитория или Space и настройки хостинга.

Планируйте около 2–3 часов, особенно при первом скачивании Python и пакетов. То, что не успели, можно завершить дома.

ПЕРЕД НАЧАЛОМ

Маршрут и требования

Этап	Что делает студент	Как проверяем
А1 · сайт	HTML/CSS-визитка, первый коммит и push	Рабочий адрес USERNAME.github.io
А2 · интерактивность	Добавляет JavaScript отдельным коммитом	Интерактивный элемент и история Git
Б1 · локально	Создаёт app.py, запускает Gradio через Python	Работающий localhost:7860 + остановка
Б2 · публикация	Переносит ту же функцию в Gradio Lite	Работающий HF Static Space
Итог	Сравнивает две архитектуры и удаляет доступы	Короткие ответы, ссылки, безопасность

Минимальные технические условия

- GitHub Pages: обычные index.html, styles.css и script.js. На первом этапе JavaScript отсутствует; сервер, React и внешние секретные API не нужны.
- На визитке есть заголовок, блок «Обо мне», минимум две карточки интересов или проектов, контакты или безопасные ссылки.
- JS меняет содержимое страницы по действию пользователя. CSS hover не считается программной интерактивностью.
- Gradio: минимум два входных параметра, работающий вывод и реакция на пустые или некорректные данные; одинаковая вычислительная функция в локальной и опубликованной версиях.
- В публичных репозиториях нет секретов, токенов, личных документов и локальных служебных файлов.

ПОЧЕМУ НЕ ПУБЛИКУЕМ ОБЫЧНЫЙ GRADIO НА HF?

По документации Hugging Face на ноябрь 2026 года Static Spaces бесплатны для всех, а создание вычислительных Gradio/Docker Spaces обычно требует платного плана. Gradio Lite позволяет продемонстрировать публикацию без платного сервера, но это другой способ исполнения Python. Сам Gradio Lite не поддерживается разработчиками с 2025 года — для курса используем как учебный пример.

ПОДГОТОВКА КОМПЬЮТЕРА

Если на Windows нет Git, редактора, Python или Gradio, откройте отдельную памятку **«Практика 01 — техническая подготовка Windows»**. Там описаны Portable Git, uv, работа без прав администратора и проверка окружения.

Первая публикация визитки

01 **Создайте HTML/CSS с помощью ИИ**

Попросите два файла — **index.html** и **styles.css** — без JS и backend. Откройте index.html двойным кликом и проверьте страницу.

ПРИМЕР ЗАПРОСА К ИИ

«Создай аккуратный адаптивный сайт-визитку: “Обо мне”, две карточки проектов, контакты. Только HTML и CSS, без JavaScript, React, CDN и сервера. Покажи код index.html и styles.css и инструкции локальной проверки».

02 **Создайте репозиторий**

На GitHub создайте **Public**-репозиторий с точным именем **USERNAME.github.io** (USERNAME — ваш логин). Не добавляйте README, .gitignore и лицензию при создании: начните с пустого репозитория.

03 **Выполните команды из папки с файлами**

Коммиты создаются на компьютере. Задайте автора **только для текущего репозитория**.

```
git init -b main
git config user.name "Student Name"
git config user.email "email@example.com"
git status
git add index.html styles.css
git commit -m "Create static homepage"
git remote add origin https://github.com/USERNAME/USERNAME.github.io.git
```

04 **Отправьте коммит**

На общем ПК используйте временный токен и отключите helper сохранения учётных данных. В HTTPS-запросе укажите логин GitHub и fine-grained PAT вместо пароля.

```
git -c credential.helper= push -u origin main
```

05 **Опубликуйте Pages**

Repository → Settings → Pages → Deploy from a branch → main → /(root) → Save. Откройте **https://USERNAME.github.io/**. Публикация может занять несколько минут.

Добавляем JavaScript новым коммитом

Теперь ваш статический сайт должен получить одну функцию, работающую **в браузере**. Действие пользователя меняет DOM, и новый запрос к Python-серверу для этого не нужен.

Идея	Что происходит
Фильтр карточек	«Все / Учёба / Хобби» показывает нужные карточки.
Переключатель темы	Меняет тему и текст кнопки.
Список целей	Добавляет запись и обновляет счётчик на странице.
Живой предпросмотр	Изменение поля ввода обновляет приветствие.

ВТОРОЙ ЗАПРОС К ИИ

«У меня уже есть index.html и styles.css. Не переписывай их с нуля. Добавь отдельный script.js с фильтром карточек, объясни, как подключить его в HTML. Учти клавиатуру и отсутствие подходящих карточек».

Проверьте локально

- Кнопка или поле работают; меняется содержание страницы, не только оформление по hover.
- После обновления страницы интерфейс по-прежнему работает; проверьте также пустые или ошибочные значения.

Второй коммит и push

```
git status
git diff
git add index.html styles.css script.js
git commit -m "Add browser-side interactivity"
git -c credential.helper= push origin main
git log --oneline -3
```

ПРОВЕРКА

На опубликованной странице появился новый элемент. В истории GitHub есть как минимум два ваших коммита. Не создавайте второй коммит через веб-редактор GitHub.

GRADIO / B1

Запускаем обычный Gradio локально

Для этого этапа нужны **Python и пакет gradio**. Их установка, в том числе без прав администратора через uv, описана в отдельной технической памятке. Она же поможет, если localhost не открывается.

01 **Подготовьте окружение**
Выполните раздел «Python / Gradio» технической памятки: создайте .venv и установите gradio. Не загружайте .venv в GitHub или Hugging Face.

02 **Создайте app.py**
Попросите ИИ написать небольшой инструмент с двумя параметрами и понятным выводом. Пример на следующей странице.

03 **Запустите локальный сервер**
Откройте терминал в папке с app.py и выполните:

```
.\.venv\Scripts\python.exe app.py
```

04 **Проверьте страницу**
Откройте **http://127.0.0.1:7860** (или адрес, который вывел Gradio). Попробуйте два разных ввода и пустое значение.

05 **Остановите сервер**
Нажмите **Ctrl+C** в терминале и обновите страницу. Она перестанет отвечать: Python выполнялся отдельным серверным процессом.

ПОДРОБНЫЕ КОМАНДЫ

Установка Python 3.12 через uv, проверка pip/Gradio, действия при блокировке приложений и диагностика запуска вынесены в отдельный PDF.

GRADIO / B1

Создаём и проверяем приложение

Можно придумать собственную задачу. Старайтесь выбирать вычисления без тяжёлых моделей и секретных API: эту же функцию затем понадобится запустить в Gradio Lite.

Идея	Входные параметры	Результат
Анализатор текста	Текст + число первых слов	Количество слов + фрагмент
Калькулятор бюджета	Доход + расходы	Остаток и пояснение
Конвертер оценок	Балл + шкала	Категория / пояснение
Тренажёр терминов	Тема + сложность	Вопрос и ответ по фиксированным правилам
Планировщик задач	Список задач + фильтр	Отобранные пункты

Работающий пример файла app.py

```
import gradio as gr

def analyze(text, limit):
    words = text.split()
    if not words:
        return 0, "Введите текст"
    n = max(1, min(30, int(limit)))
    return len(words), " ".join(words[:n])

demo = gr.Interface(
    fn=analyze,
    inputs=[
        gr.Textbox(label="Текст", lines=4),
        gr.Slider(1, 30, value=5, step=1, label="Сколько слов")
    ],
    outputs=[gr.Number(label="Слов"),
             gr.Textbox(label="Первые слова")],
    title="Мини-анализатор текста"
)
demo.launch()
```

ПРОВЕРКА

Введите текст с пробелами, поменяйте значение слайдера, отправьте пустую строку. Убедитесь, что оба параметра влияют на результат, а программа не падает. Затем остановите процесс сочетанием Ctrl+C.

СРАВНИВАЕМ АРХИТЕКТУРУ

Что происходит при нажатии кнопки?

Для сравнения используйте **одну и ту же** вычислительную функцию. Интерфейс может быть описан схожими строками gr.Interface(...), однако место исполнения Python различается.

ОБЫЧНЫЙ GRADIO — ЛОКАЛЬНЫЙ СЕРВЕР

Браузер → HTTP-соединение с 127.0.0.1:7860 → сервер Gradio → Python-функция → ответ → интерфейс.
Frontend Gradio уже существует как готовый набор браузерных ресурсов. Функция analyze() выполняется в отдельном Python-процессе, а не в браузере.

GRADIO LITE — PYTHON В БРАУЗЕРЕ

Браузер загружает HTML/JS/Pyodide → функция Python выполняется через WebAssembly в браузере → обновляется интерфейс.
Отдельный Python-сервер для обработки действия не требуется. Код Python не переводится в JavaScript: работает интерпретатор Python, собранный для WebAssembly.

Характеристика	Обычный Gradio	Gradio Lite
Где Python	На сервере / localhost	В браузере посетителя
Хостинг	Нужен Python-процесс	Достаточно Static Space
Остановка процесса	Приложение перестаёт работать	Не зависит от локального app.py
Библиотеки	Пакеты для обычного Python	Только совместимые с Pyodide
Секреты	Можно хранить на сервере безопасно	Нельзя хранить в браузере

Ответьте на вопросы

- В какой версии Python выполняется в браузере, а в какой — в отдельном процессе?
- Почему локальная версия перестаёт работать после Ctrl+C, а опубликованная Lite-версия — нет?
- Если хотите изучить запросы и загрузку Pyodide через DevTools, воспользуйтесь отдельной технической памяткой.

ВАЖНО

Gradio Lite был архивирован в 2025 году. Он используется здесь исключительно для учебного сравнения архитектур и бесплатной статической публикации. Не стоит выбирать его по умолчанию для долгосрочного проекта.

Публикуем ту же функцию в Static Space

01 **Создайте новый Space**

Зайдите в Hugging Face → New Space. Укажите имя, видимость Public, SDK **Static** (не Gradio и не Docker). Создать пустой Space через браузер разрешено, но все файлы дальше редактируются локально.

02 **Склонируйте Space**

В терминале подставьте свой HF-логин и название Space.

```
git clone https://huggingface.co/spaces/HF_USER/SPACE_NAME
cd SPACE_NAME
git config user.name "Student Name"
git config user.email "email@example.com"
```

03 **Настройте README.md**

В самом начале файла должна быть YAML-шапка:

```
---
title: My Gradio Tool
sdk: static
app_file: index.html
---
```

04 **Создайте index.html**

Возьмите шаблон с следующей страницы. Код функции **analyze** и описание интерфейса перенесите из локального app.py без изменения логики. Минимальная учебная версия допускает прямо встроенный Python.

05 **Проверьте состав папки**

В Space должны лежать **README.md** и **index.html**. Не загружайте в Space **uv.exe**, **.venv**, содержимое кэшей и временные токены.

ПОЧЕМУ НЕ SDK GRADIO?

На ноябрь 2026 года HF Static Space бесплатен для всех. Обычный Gradio/Docker Space запускает сервер и имеет ограничения по правам на создание. Lite здесь позволяет опубликовать браузерную версию без оплачиваемого сервера.

Обратите внимание: тег gradio-file внутри index.html — способ передать Python-код в Gradio Lite. Файл приложения на HF в этом упражнении статический, хотя внутри выполняется Python.

Рабочий шаблон index.html

Пример ниже повторяет функцию из локального app.py. В собственном проекте замените её и параметры интерфейса на свои. Сохраняйте UTF-8.

```
<!doctype html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <script type="module" src="https://cdn.jsdelivr.net/npm/@gradio/lite/dist/lite.js"></script>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@gradio/lite/dist/lite.css">
</head>
<body>
<gradio-lite>
<gradio-file name="app.py" entrypoint>
import gradio as gr

def analyze(text, limit):
    words = text.split()
    if not words:
        return 0, "Введите текст"
    n = max(1, min(30, int(limit)))
    return len(words), " ".join(words[:n])

demo = gr.Interface(
    fn=analyze,
    inputs=[
        gr.Textbox(label="Текст", lines=4),
        gr.Slider(1, 30, value=5, step=1, label="Сколько слов")
    ],
    outputs=[gr.Number(label="Слов"),
             gr.Textbox(label="Первые слова")],
    title="Мини-анализатор текста"
)
demo.launch()
</gradio-file>
</gradio-lite>
</body>
</html>
```

Пример использует CDN-версию Gradio Lite. Сам проект Lite больше не поддерживается; если CDN/совместимость пакетов изменится, обратитесь к технической памятке или преподавателю.

Push и проверка

01 **Зафиксируйте локальные файлы**

Убедитесь, что в Space нет ключей, локальной .venv и других служебных файлов.

```
git status
git add README.md index.html
git commit -m "Publish browser-based Gradio app"
git -c credential.helper= push origin main
```

02 **Авторизуйтесь безопасно**

При HTTPS-авторизации укажите HF-логин и временный Hugging Face User Access Token с правом записи для нужного Space, если доступно такое ограничение. Токен вводится в скрытый запрос терминала, не в команду.

03 **Откройте опубликованную версию**

Адрес **https://huggingface.co/spaces/HF_USER/SPACE_NAME** → вкладка App. Первая загрузка Python через Pyodide может быть долгой; дождитесь появления интерфейса.

04 **Проверьте одинаковую функцию**

Подайте одинаковые значения в локальном Gradio (если сервер запущен) и опубликованном Lite. Для учебного анализатора должны совпадать оба результата.

ЕСЛИ ПРИЛОЖЕНИЕ НЕ ЗАПУСКАЕТСЯ

Проверка Network/Console, загрузки lite.js и Pyodide, проблем с 403 и портом 7860 подробно описана в технической памятке.

БЕЗОПАСНОСТЬ / ОБЯЗАТЕЛЬНО

Отзовите токены перед уходом

Если вы работаете на общем университетском компьютере, необходимо не только закончить push, но и **отозвать временные токены доступа**. Это обязательный шаг практики.

01 **Не сохраняйте пароль или токен**
Для GitHub и Hugging Face используйте токены с минимальными правами, не вставляйте их в команды или URL. Выполняйте **git -c credential.helper= push** — команда отключает credential helper для этого запуска Git.

02 **Отзовите GitHub-токен**
GitHub → Settings → Developer settings → Personal access tokens. Найдите токен, созданный для занятия, и выберите **Revoke/Delete** (название зависит от типа токена).

03 **Отзовите Hugging Face-токен**
Hugging Face → Settings → Access Tokens. Найдите временный токен для отправки в Space и отзовите/удалите его.

04 **Завершите сеанс**
Выйдите из GitHub и Hugging Face, закройте окна браузера, удалите рабочие файлы на общем ПК. Если авторизация ранее была сохранена, очистите её в Windows Credential Manager. Не используйте постоянные SSH-ключи на чужом компьютере.

ВАЖНО

Отзыв токенов обязателен, даже если вы удалили папку проекта и закрыли браузер.
Закрытие приватного окна не делает сохранённый где-либо токен недействительным.

Подробный чек-лист безопасности и действия для чистой Windows есть в отдельной технической памятке.

ПРИЁМКА

Что сдавать и как оценивается

Одно сообщение преподавателю со ссылками на работающие проекты, историей изменений и кратким объяснением архитектуры.

```
GitHub repo: https://github.com/USERNAME/USERNAME.github.io
Pages:       https://USERNAME.github.io/
HF Space:    https://huggingface.co/spaces/HF_USER/SPACE_NAME
2 commits:   <SHA начальной версии> / <SHA JS-версии>
```

Кроме ссылок: покажите запуск app.py локально (при очной проверке) или приложите скриншот терминала с адресом localhost и страницу интерфейса; продемонстрируйте его остановку и объясните 4–6 предложениями разницу с опубликованным Lite.

Критерий	Проверка	Баллы
Статический сайт	Первая версия Pages и локальный commit + push	20
JavaScript	Реакция на действие + отдельный второй commit	20
Локальный Gradio	Запуск app.py, два параметра, корректный результат, остановка	25
HF Static Space	Та же функция в Lite, рабочая публичная ссылка	20
Архитектура / безопасность	Объяснение, отсутствие секретов, отзыв токенов	15

Ключевые вопросы

- Что именно перестаёт работать после Ctrl+C у локального Gradio и почему?
- Где исполняется функция analyze() в каждом варианте?
- Почему Lite можно разместить как Static Space, а обычный Gradio требует Python-сервера?
- Каким способом вы отправили код и как обезопасили общий компьютер?

Официальные материалы

Gradio: https://www.gradio.app/guides/quickstart · HF Spaces: https://huggingface.co/docs/hub/spaces-overview
Gradio Lite: https://github.com/gradio-app/gradio-lite · техническая памятка — отдельный PDF в папке занятия.